

Débora Cristina Pereira Marra (12411ECP017), Gabriel Martins Santos Almeida (12411ECP012), Jose Lucas Lira Bizil (12411ECP005), Nycholas Rodrigues Cunha Coelho (12521ECP031), Vitor Augusto Machado Silami (12411ECP019)

Rede Social Simplificada

Uberlândia – MG

Julho de 2026

SUMÁRIO

1	INTRODUÇÃO	2
2	MÉTODOS	4
2.1	Arquitetura geral	4
2.2	Modelagem dos TADs	4
2.2.1	Árvore AVL (índice por id)	4
2.2.2	Tabela hash (índice por login e por nome)	5
2.2.3	Lista, fila e pilha	5
2.2.4	Grafo de amizades	5
3	RESULTADOS	7
3.1	Complexidade das operações	7
3.2	Capturas de execução dos casos de teste	7
3.2.1	I. Cadastro de usuário	8
3.2.2	II. Busca de usuário por nome	8
3.2.3	III. Adicionar amizade	8
3.2.4	IV. Remover amizade	9
3.2.5	V. Criar publicação	9
3.2.6	VI. Exibir publicações	9
3.2.7	VII. Histórico (N=10, mais recente primeiro)	9
3.2.8	VIII. Exibir amigos	10
3.2.9	IX. Verificar conexão entre usuários	10
3.2.10	X. Sugestão de amizades	10
3.2.11	Requisito não funcional I: persistência	10
3.2.12	Requisitos não funcionais II e III: validação e duplicidade	11
	Referências	12

1 INTRODUÇÃO

Redes sociais digitais armazenam e relacionam grandes volumes de dados de usuários, conexões e conteúdo publicado, exigindo estruturas de dados capazes de sustentar operações de busca, inserção e travessia com desempenho previsível à medida que a base de usuários cresce. Este relatório descreve o desenvolvimento de uma *rede social simplificada*, implementada em linguagem C, como Projeto Final da disciplina de Estruturas de Dados (Tema 04), contemplando cadastro e busca de usuários, gestão de amizades, publicações, histórico de atividades e sugestão de novas conexões.

Um Tipo Abstrato de Dados (TAD) define um conjunto de dados e as operações permitidas sobre eles, independentemente de sua implementação interna (CORMEN et al., 2012). A escolha do TAD adequado a cada funcionalidade determina diretamente a complexidade assintótica das operações do sistema, normalmente expressa pela notação $O(n)$: $O(1)$ para operações de custo constante, $O(\log n)$ para buscas em estruturas balanceadas e $O(n)$ para percursos lineares (ZIVIANI, 2011).

No sistema desenvolvido, seis TADs clássicos foram empregados, cada um resolvendo um subproblema específico:

1. uma **árvore binária de busca balanceada (AVL)** indexa os usuários pelo identificador (*id*), garantindo busca em $O(\log n)$ mesmo no pior caso;
2. uma **tabela hash** com tratamento de colisões por encadeamento separado indexa os usuários por *login* e por nome, com custo médio $O(1)$, realocando (*rehash*) seus baldes dinamicamente para preservar esse desempenho;
3. uma **lista encadeada** genérica armazena as publicações de cada usuário e as listas de adjacência do grafo;
4. uma **fila** (FIFO) sustenta a busca em largura (*BFS*) usada para verificar conectividade e gerar sugestões de amizade;
5. uma **pilha** (LIFO) mantém o histórico das atividades mais recentes, sempre exibido da mais nova para a mais antiga;
6. um **grafo** não direcionado representa a rede de amizades entre os usuários.

O objetivo deste trabalho é apresentar o projeto, a modelagem e os resultados obtidos na implementação de um sistema de rede social simplificada em C, atendendo às dez funcionalidades obrigatórias e aos três requisitos não funcionais especificados no

enunciado do Projeto Final – Tema 04, com ênfase na justificativa do uso correto e eficiente de cada estrutura de dados exigida.

2 MÉTODOS

2.1 Arquitetura geral

O sistema é organizado em módulos independentes, cada TAD implementado como um par de arquivos `.h/.c`, conforme exigido no enunciado. O arquivo `main.c` é responsável apenas pela interface de menu (E/S com o usuário) e não contém regra de negócio; toda a lógica de cadastro, validação e integração entre as estruturas está concentrada no módulo de fachada `rede_social`, que mantém a árvore AVL, as duas tabelas hash, o grafo de amigos e a pilha de histórico, delegando a cada TAD apenas a responsabilidade que lhe é própria. O módulo `persistencia` lê e grava o estado da fachada em um arquivo de texto (`rede_social.dat`), sem que os demais módulos precisem conhecer o formato de armazenamento. A Figura 1 ilustra essa arquitetura.

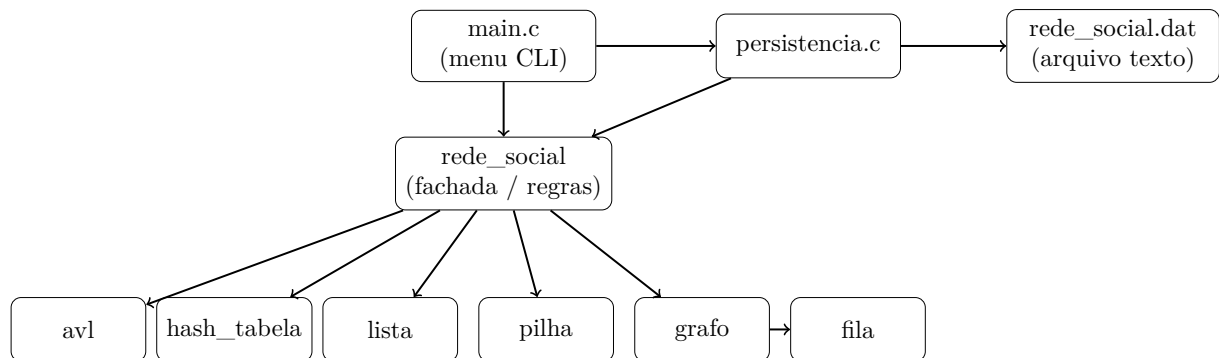


Figura 1 – Arquitetura de módulos do sistema

Fonte: elaborado pelos autores.

2.2 Modelagem dos TADs

2.2.1 Árvore AVL (índice por id)

A árvore AVL indexa cada `Usuario` pelo campo `id`, atribuído automaticamente e de forma sequencial no cadastro. Cada nó armazena o `id`, um ponteiro para o usuário e sua altura, recalculada a cada inserção; quando o fator de balanceamento de um nó ultrapassa 1 em módulo, uma das quatro rotações clássicas (simples à esquerda, simples à direita, dupla esquerda-direita ou dupla direita-esquerda) é aplicada. Como o enunciado não exige remoção de usuários – apenas de amigos –, a árvore implementa somente inserção, busca e percurso em ordem (usado pela persistência), o que simplifica a implementação sem abrir

mão da garantia de $O(\log n)$ mesmo quando os *ids* são inseridos em sequência crescente, caso em que uma BST comum degeneraria em uma lista.

2.2.2 Tabela hash (índice por login e por nome)

Duas instâncias de uma mesma tabela hash genérica (chave textual $\rightarrow$ ponteiro) indexam os usuários: uma por **login**, chave única usada para impedir cadastros duplicados, e outra por **nome**, que aceita múltiplos usuários por chave (acumulados em uma lista encadeada) para suportar a busca por nome. Colisões são tratadas por encadeamento separado (Figura 2); a função de espalhamento utilizada é a *djb2*. Para preservar o custo médio $O(1)$ mesmo com o crescimento da base de usuários, a tabela dobra o número de baldes e redistribui todas as entradas sempre que o fator de carga ultrapassa 0,75.

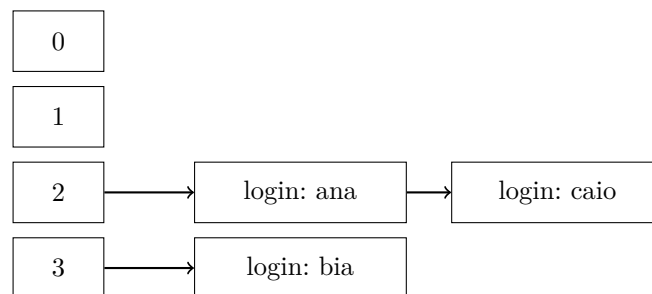


Figura 2 – Tabela hash com tratamento de colisões por encadeamento separado

Fonte: elaborado pelos autores. Baldes 0 e 1 vazios; balde 2 com colisão de duas chaves distintas.

2.2.3 Lista, fila e pilha

A lista encadeada é implementada de forma genérica (armazena `void*`) e reaproveitada em dois papéis: como lista de publicações de cada usuário – inserção sempre no início, de modo que a publicação mais recente apareça primeiro – e como lista de adjacência do grafo de amizades. A fila (FIFO) sustenta a busca em largura (*BFS*) usada tanto para verificar conectividade quanto para gerar sugestões de amizade. A pilha (LIFO) registra o histórico das $N = 10$ atividades mais recentes do sistema (novo usuário, nova amizade, remoção de amizade e nova publicação); ao ultrapassar essa capacidade, a atividade mais antiga é descartada automaticamente, e o percurso do topo para a base entrega os eventos da mais recente para a mais antiga, exatamente como pedido no enunciado.

2.2.4 Grafo de amizades

O grafo de amizades é não direcionado e implementado de forma desacoplada da `struct Usuario`: seus vértices são simplesmente os *ids* dos usuários, e cada vértice

mantém uma lista de adjacência com os *ids* de seus amigos diretos. Essa independência permite que o grafo seja testado isoladamente, sem depender de nenhuma outra estrutura do sistema. Duas operações relevantes utilizam busca em largura (*BFS*): a **verificação de conexão** entre dois usuários, que percorre o grafo a partir da origem até encontrar o destino ou esgotar os vértices alcançáveis; e a **sugestão de amizades**, que identifica os vértices a exatamente distância 2 da origem (amigos de amigos, excluindo os amigos diretos e o próprio usuário), conta quantos amigos em comum cada candidato tem com a origem e ordena o resultado por esse número de forma decrescente, com empates desfeitos em ordem alfabética pelo nome – limitando a saída a três sugestões, como exigido. A Figura 3 ilustra o cenário usado no caso de teste de sugestão de amizades (Seção 3.2, item X).

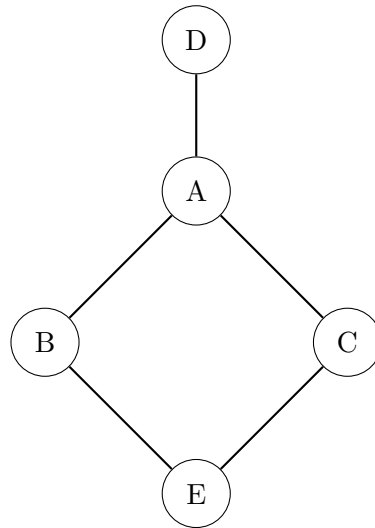


Figura 3 – Exemplo de grafo de amizades usado no caso de teste de sugestão

Fonte: elaborado pelos autores. A busca em largura a partir de E alcança A em profundidade 2 (via B e via C), tornando A a única sugestão, com 2 amigos em comum.

3 RESULTADOS

3.1 Complexidade das operações

A Tabela 1 resume a complexidade assintótica das principais operações de cada TAD implementado, e a Tabela 2 mapeia essa complexidade para cada funcionalidade obrigatória do enunciado. Em ambas, n é o número de usuários cadastrados, V e E são o número de vértices e arestas do grafo de amizades, $N = 10$ é o tamanho fixo do histórico e “grau” é o número de amigos diretos de um usuário.

Tabela 1 – Complexidade das operações por TAD

Estrutura	Operação	Complexidade	Observação
Árvore AVL	Inserir	$O(\log n)$	Balanceada; nunca degenera
Árvore AVL	Buscar por id	$O(\log n)$	
Tabela hash	Inserir / buscar	$O(1)$ médio	<i>Rehash</i> ao superar fator de carga 0,75
Lista	Inserir (início/fim)	$O(1)$	Mantém ponteiros início/fim
Lista	Buscar / remover valor	$O(k)$	k = tamanho da lista
Fila	(Des)enfileirar	$O(1)$	
Pilha	Empilhar	$O(1)$	Descarta o mais antigo se $> N$
Pilha	Percorrer histórico	$O(N)$	$N = 10$, constante
Grafo	Adicionar/remover aresta	$O(\text{grau})$	Verifica duplicidade antes
Grafo	Verificar conexão (BFS)	$O(V + E)$	
Grafo	Sugerir amigos (BFS + contagem)	$O(V + E + \text{grau}^2)$	Pior caso

Fonte: elaborado pelos autores, a partir da implementação em `src/`.

3.2 Capturas de execução dos casos de teste

Esta seção reproduz, para cada funcionalidade obrigatória, a saída real obtida ao executar o binário `bin/rede_social` com os cenários descritos em `tests/casos_teste.md`. As capturas abaixo são transcrições literais do terminal (*stdout*), não simulações.

Tabela 2 – Complexidade dominante por funcionalidade obrigatória

Funcionalidade	Estruturas envolvidas	Complexidade
I. Cadastro de usuário	AVL + Hash (login/nome) + Grafo	$O(\log n)$
II. Busca por nome	Tabela hash (nome)	$O(1)$ médio + $O(k)$
III. Adicionar amizade	Grafo	$O(\text{grau})$
IV. Remover amizade	Grafo	$O(\text{grau})$
V. Criar publicação	AVL (localizar) + Lista	$O(\log n)$
VI. Exibir publicações	Lista	$O(k)$
VII. Histórico	Pilha	$O(N)$
VIII. Exibir amigos	Grafo + AVL	$O(\text{grau} \times \log n)$
IX. Verificar conexão	Grafo (BFS)	$O(V + E)$
X. Sugestão de amizades	Grafo (BFS)	$O(V + E + \text{grau}^2)$

Fonte: elaborado pelos autores. k representa o número de itens retornados (amigos, publicações ou resultados de busca).

3.2.1 I. Cadastro de usuário

```
Opcao: 1
Nome: Ana Souza
Login: ana
Usuario cadastrado com sucesso! id = 1
```

Caso de erro (login duplicado):

```
Opcao: 1
Nome: Outro Nome
Login: ana
Erro: Ja existe um usuario cadastrado com esse login.
```

3.2.2 II. Busca de usuário por nome

```
Opcao: 2
Nome a buscar: Ana Souza
Usuarios encontrados:
- id 1 | nome: Ana Souza | login: ana
```

Caso de erro (nome inexistente):

```
Opcao: 2
Nome a buscar: Ninguem Assim
Nenhum usuario encontrado com o nome "Ninguem Assim".
```

3.2.3 III. Adicionar amizade

```
Opcao: 3
Id do primeiro usuario: 1
Id do segundo usuario: 2
Amizade criada com sucesso.
```

Caso de erro (amizade duplicada):

```
Opcao: 3
Id do primeiro usuario: 1
Id do segundo usuario: 2
Erro: Esses usuarios ja sao amigos.
```

3.2.4 IV. Remover amizade

```
Opcao: 4
Id do primeiro usuario: 1
Id do segundo usuario: 2
Amizade removida com sucesso.
```

Caso de erro (remoção de amizade inexistente):

```
Opcao: 4
Id do primeiro usuario: 1
Id do segundo usuario: 2
Erro: Esses usuarios nao sao amigos.
```

3.2.5 V. Criar publicação

```
Opcao: 5
Id do usuario: 1
Texto da publicacao: Ola, mundo!
Publicacao criada com sucesso.
```

3.2.6 VI. Exibir publicações

```
Opcao: 6
Id do usuario: 1
Publicacoes de Ana Souza (mais recente primeiro):
- Ola, mundo!
```

3.2.7 VII. Histórico (N=10, mais recente primeiro)

```
Opcao: 7
```

```
Historico de atividades (mais recente primeiro):
```

```
[Nova publicacao] Nova publicacao de Ana Souza (id 1)
[Nova amizade] Nova amizade: Ana Souza (id 1) e Carla Dias (id 3)
[Nova amizade] Nova amizade: Ana Souza (id 1) e Bruno Lima (id 2)
[Novo usuario] Novo usuario cadastrado: Carla Dias (id 3, login carla)
[Novo usuario] Novo usuario cadastrado: Bruno Lima (id 2, login bruno)
[Novo usuario] Novo usuario cadastrado: Ana Souza (id 1, login ana)
```

3.2.8 VIII. Exibir amigos

```
Opcao: 8
```

```
Id do usuario: 1
```

```
Amigos de Ana Souza:
```

```
- id 2 | nome: Bruno Lima | login: bruno
- id 3 | nome: Carla Dias | login: carla
```

3.2.9 IX. Verificar conexão entre usuários

```
Opcao: 9
```

```
Id do primeiro usuario: 1
```

```
Id do segundo usuario: 2
```

```
Os usuarios 1 e 2 estao conectados na rede.
```

3.2.10 X. Sugestão de amizades

```
Opcao: 10
```

```
Id do usuario: 3
```

```
Sugestoes de amizade para Carla Dias:
```

```
- Bruno Lima (id 2) | 1 amigo(s) em comum
```

3.2.11 Requisito não funcional I: persistência

O script `tests/smoke_test.sh` automatiza a verificação deste requisito: executa o binário três vezes em sequência, em um diretório isolado, cadastrando usuários, amizade e publicação na primeira execução e confirmando – via busca por nome, exibição de amigos e exibição de publicações – que todos os dados persistiram nas execuções seguintes, inclusive que o próximo *id* gerado continua a sequência anterior (não reinicia em 1). A execução real do script produz:

```
Smoke test de persistencia: OK (3 execucoes sucessivas, dados preservados)
```

3.2.12 Requisitos não funcionais II e III: validação e duplicidade

Todas as funcionalidades tratam entradas inválidas ou inexistentes sem encerrar o programa abruptamente – ver os casos de erro já reproduzidos nas Seções 3.2.1 a 3.2.4 e 3.2.9 (login duplicado, nome sem resultado, amizade duplicada, remoção de amizade inexistente e ids inexistentes). Além disso, entradas não numéricas no menu (letras, linhas vazias) são detectadas e o usuário é solicitado a digitar novamente, como mostrado abaixo:

```
Opcao: abc
Entrada invalida. Digite um numero inteiro.
Opcao:
```

Essas verificações também são cobertas de forma automatizada em `tests/test_rede_social.c` (fachada) e nos testes unitários de cada TAD, totalizando 90 asserções executadas via `make test`, todas aprovadas, com compilação limpa (`gcc -Wall -Wextra -std=c11`) e sem nenhum aviso do compilador.

REFERÊNCIAS

CORMEN, Thomas H. et al. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

ZIVIANI, Nivio. **Projeto de algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.